

CIVL-ized Concurrent Programs

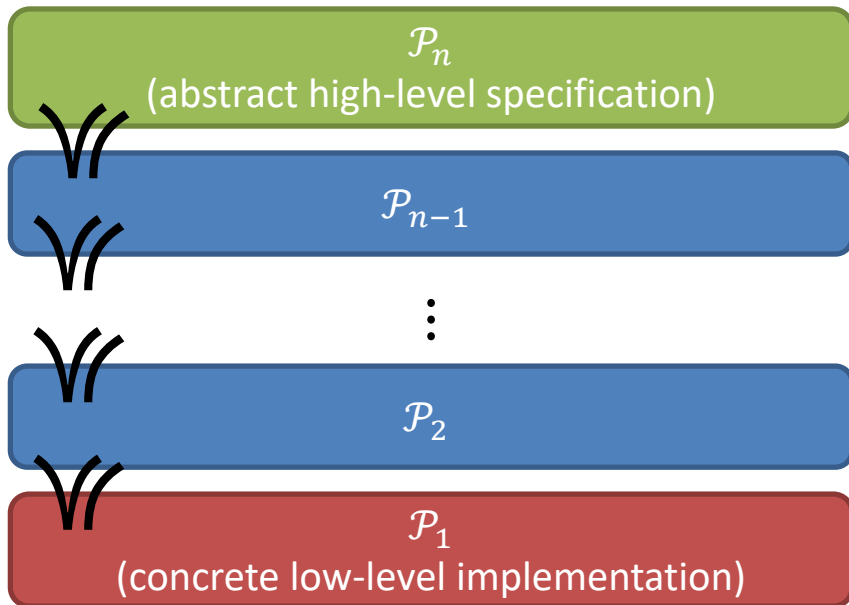
Bernhard Kragl

IST Austria

joint work with Shaz Qadeer

CIVL: Layered Refinement over Structured Concurrent Programs

Layered Concurrent Program [CAV'18]



Procedures

```
proc P(...) { S }  
  
 $S_1 ; S_2$    if * then  $S_1$  else  $S_2$    exec A  
call P   call  $P_1 \parallel P_2$    async P   ...
```

Gated atomic actions

$\text{CAS}_x(\text{old}, \text{new})$	$\text{RELEASE}(\text{tid})$
returns (s)	assert $l = \text{Some}(\text{tid})$
$s := (x = \text{old})$	$l := \text{None}$
if s then $x := \text{new}$	

In this talk

- Verification features (layered proofs)
- Programmer interaction (layered programs)
- Implementation atop Boogie

var x: int

action INC()

x := x + 1

hidden local

var x: int
var l: Option<Tid>

introduced global

hidden global

proc Inc(*linear* tid)

exec ACQUIRE(tid)
exec t := READ(tid)
exec WRITE(tid, t+1)
exec RELEASE(tid)

rewritten calls

introduced local

introduced local

action ACQUIRE(*linear* tid)

assume l = None
l := Some(tid)

right mover

action RELEASE(*linear* tid)

assert l = Some(tid)
l := None

left mover

action READ(*linear* tid)

returns (v)

assert l = Some(tid)

v := x

both mover

action WRITE(*linear* tid, v)

assert l = Some(tid)

x := v

both mover

Invariant: $b \Leftrightarrow l \neq \text{None}$

var x: int
var b: bool

hidden global

proc Inc()

call Acquire()
call t := Read()
call Write(t+1)
call Release()

proc Acquire()

exec s := CAS_b(false, true)
if (¬s) **then** **call** Acquire()
else intro [l := Some(tid)]

proc Release()

exec [b := false]
intro [l := None]

introduction
action

proc Read()

returns (v)
exec [v := x]

proc Write(v)

exec [x := v]

From layer $\mathcal{P}_i$ to $\mathcal{P}_{i+1}$

- Preemptive to cooperative semantics on $\mathcal{P}_i$
 - Actions: mover types & commutativity checks
 - Procedures: simulation check ($R^*N^?L^*$) on yield-to-yield fragments

proc Inc(*linear* tid)

exec ACQUIRE(tid)
exec t := READ(tid)
exec WRITE(tid, t+1)
exec RELEASE(tid)

action ACQUIRE(*linear* tid)

assume l = None

l := Some(tid) right mover

action RELEASE(*linear* tid)

assert l = Some(tid)

l := None left mover

action READ(*linear* tid)

returns (v)

assert l = Some(tid)

v := x both mover

action WRITE(*linear* tid, v)

assert l = Some(tid)

x := v both mover

From layer $\mathcal{P}_i$ to $\mathcal{P}_{i+1}$

- Preemptive to cooperative semantics on $\mathcal{P}_i$
 - Actions: mover types & commutativity checks
 - Procedures: simulation check ($R^*N^?L^*$) on yield-to-yield fragments
- Instrument $\mathcal{P}_i$ with introduced state $\rightarrow \widetilde{\mathcal{P}}_i$

```
proc Acquire(linear tid)  
  exec s := CASb(false, true)  
  if (¬s) then call Acquire()  
  else intro [l := Some(tid)]
```

```
proc Release()  
  exec [b := false]  
  intro [l := None]
```



introduction
action

From layer $\mathcal{P}_i$ to $\mathcal{P}_{i+1}$

- Preemptive to cooperative semantics on $\mathcal{P}_i$
 - Actions: mover types & commutativity checks
 - Procedures: simulation check ($R^*N^?L^*$) on yield-to-yield fragments
- Instrument $\mathcal{P}_i$ with introduced state $\rightarrow \widetilde{\mathcal{P}}_i$
- Modularly check that some procedures P in $\widetilde{\mathcal{P}}_i$ are summarized by atomic actions A
 - Establish context $\rightarrow$ invariants (location invariants, yield invariants)
 - Every execution (i.e., path) through P behaves like $SKIP^*; A; SKIP^*$



From layer $\mathcal{P}_i$ to $\mathcal{P}_{i+1}$

- Preemptive to cooperative semantics on $\mathcal{P}_i$
 - Actions: mover types & commutativity checks
 - Procedures: simulation check ($R^*N^?L^*$) on yield-to-yield fragments
- Instrument $\mathcal{P}_i$ with introduced state $\rightarrow \widetilde{\mathcal{P}}_i$
- Modularly check that some procedures P in $\widetilde{\mathcal{P}}_i$ are summarized by atomic actions A
 - Establish context $\rightarrow$ invariants (location invariants, yield invariants)
 - Every execution (i.e., path) through P behaves like $SKIP^*; A; SKIP^*$

CAV talk tomorrow



From layer $\mathcal{P}_i$ to $\mathcal{P}_{i+1}$

- Preemptive to cooperative semantics on $\mathcal{P}_i$
 - Actions: mover types & commutativity checks
 - Procedures: simulation check ($R^*N^?L^*$) on yield-to-yield fragments
- Instrument $\mathcal{P}_i$ with introduced state $\rightarrow \widetilde{\mathcal{P}}_i$
- Modularly check that some procedures P in $\widetilde{\mathcal{P}}_i$ are summarized by atomic actions A
 - Establish context $\rightarrow$ invariants (location invariants, yield invariants)
 - Every execution (i.e., path) through P behaves like $SKIP^*; A; SKIP^*$
- In $\widetilde{\mathcal{P}}_i$, replace P with $A \rightarrow \mathcal{P}_{i+1}$
 - Drop hidden state

var x: int

action INC()

x := x + 1

var x: int
var l: Option<Tid>

hidden global

proc Inc(*linear* tid)
 exec ACQUIRE(tid)
 exec t := READ(tid)
 exec WRITE(tid, t+1)
 exec RELEASE(tid)

rewritten calls

hidden local

action ACQUIRE(*linear* tid)
 assume l = None
 l := Some(tid)

action RELEASE(*linear* tid)
 assert l = Some(tid)
 l := None

action READ(*linear* tid)
returns (v)
 assert l = Some(tid)
 v := x

action WRITE(*linear* tid, v)
 assert l = Some(tid)
 x := v

Invariant: $b \Leftrightarrow l \neq \text{None}$

var x: int
var b: bool

hidden global

proc Inc()
 call Acquire()
 call t := Read()
 call Write(t+1)
 call Release()

proc Acquire()
 exec s := CAS_b(false, true)
 if (¬s) **then** **call** Acquire()
 else intro [l := Some(tid)]

proc Release()
 exec [b := false]
 intro [l := None]

proc Read()
returns (v)
 exec [v := x]

proc Write(v)
 exec [x := v]

From layer $\mathcal{P}_i$ to $\mathcal{P}_{i+1}$

- Preemptive to cooperative semantics on $\mathcal{P}_i$
 - Actions: mover types & commutativity checks
 - Procedures: simulation check ($R^*N^?L^*$) on yield-to-yield fragments
- Instrument $\mathcal{P}_i$ with introduced state $\rightarrow \widetilde{\mathcal{P}}_i$
- Modularly check that some procedures P in $\widetilde{\mathcal{P}}_i$ are summarized by atomic actions A
 - Establish context $\rightarrow$ invariants (location invariants, yield invariants)
 - Every execution (i.e., path) through P behaves like $SKIP^*; A; SKIP^*$
- In $\widetilde{\mathcal{P}}_i$, replace P with $A \rightarrow \mathcal{P}_{i+1}$
 - Drop hidden state

Demo

- Program 1-3
 - Invariants
 - Movers
 - Refinement
- Lock example
 - Layered concurrent programs

Implementation of CIVL atop Boogie

- Input: Layered concurrent program $\mathcal{LP}$
 - Layer type checking
- Extract concurrent programs $\mathcal{P}_1, \dots, \mathcal{P}_n$ from $\mathcal{LP}$
- For every procedure P in $\mathcal{P}_i$
 - Yield sufficiency check (reduction)
 - Explicit interference: havoc global state and assume invariant at every yield point
 - Instrument noninterference checking
 - Instrument refinement checking
- “On the side”
 - Commutativity checking (mover types)
 - Linearity checking $\rightarrow$ free invariants
- Send sequential Boogie procedures to Z3

Demo

Desugaring CIVL program into Boogie

```
boogie /civlDesugaredFile:des.bpl lock.bpl
```

Tool and Examples



github.com/boogie-org/boogie

- Concurrent garbage collector [Hawblitzel et. al; CAV'15]
- VerifiedFT race detector [Flanagan et. al; PPOPP'18]
- Weak memory (TSO) programs [Bouajjani, et. al; CAV'18]
- Chase-Lev deque [Mutluergil & Tasiran; Computing '19]
- Weakly-consistent data structures [Krishna et. al; ESOP'20]
- Two-phase commit [Kragl et. al; CONCUR'18]
- Paxos [Kragl et. al; PLDI'20]
- Lock-protected memory atop TSO, thread-local heap atop shared heap, work-stealing queue, Treiber stack, ticket lock, ...

Related Work

Refinement

TLA+, Event-B,
CertiKOS/CCAL, CSPEC, IronFleet, Armada, ...

Concurrency Verifiers

Chalice, VCC, VeriFast, VerCors, Viper, Verdi, ...

Software Model Checking

Blast, Threader, Weaver, ...